

# Universidade Federal do Rio de Janeiro



Universidade Federal  
do Rio de Janeiro  

---

Escola Politécnica

## Projeto de uma Unidade Lógica e Aritmética (ULA) de 4 bits em VHDL

Sistemas Digitais

|             |                                               |
|-------------|-----------------------------------------------|
| Alunos      | Daniel Gudin<br>João Gabriel<br>Rayane Santos |
| Professor   | Pedro Cruz                                    |
| Instituição | Universidade Federal do Rio de Janeiro (UFRJ) |

Rio de Janeiro, 19 de Maio de 2026

# Sumário

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                                      | <b>1</b>  |
| <b>2</b> | <b>Diagrama da Máquina de Estados (FSM)</b>                            | <b>1</b>  |
| 2.1      | Descrição dos Estados da FSM . . . . .                                 | 1         |
| <b>3</b> | <b>Diagrama de Blocos Estrutural)</b>                                  | <b>3</b>  |
| 3.1      | Descrição do Diagrama de Blocos Estrutural (Datapath e Controle) . . . | 3         |
| <b>4</b> | <b>Circuito de Debounce</b>                                            | <b>5</b>  |
| <b>5</b> | <b>Simulações</b>                                                      | <b>5</b>  |
| 5.1      | Teste 1: Adição (ADD) forçando um Overflow (V) . . . . .               | 5         |
| 5.2      | Teste 2: Subtração (SUB) forçando a Flag Zero (Z) . . . . .            | 6         |
| 5.3      | Teste 3: Incremento (INC) testando o Pulo da FSM . . . . .             | 7         |
| 5.4      | Teste 4: Porta Lógica OR . . . . .                                     | 8         |
| 5.5      | Teste 5: Porta Lógica AND . . . . .                                    | 9         |
| 5.6      | Teste 6: Porta Lógica XOR (Ou Exclusivo) . . . . .                     | 10        |
| 5.7      | Teste 7: Negação (NEG) forçando Overflow . . . . .                     | 11        |
| 5.8      | Teste 8: Shift Left (SHL) testando a Flag Carry (C) . . . . .          | 12        |
| <b>6</b> | <b>Conclusão</b>                                                       | <b>13</b> |
| <b>7</b> | <b>Referências</b>                                                     | <b>14</b> |

## Resumo

Este trabalho apresenta o desenvolvimento de uma Unidade Lógica e Aritmética (ULA) de 4 bits implementada em VHDL para a plataforma FPGA Xilinx Spartan-3. O sistema utiliza uma máquina de estados finitos (FSM) para controlar a entrada de dados por meio de chaves e botão, incluindo tratamento de debounce. A ULA realiza oito operações aritméticas e lógicas, gerando flags de status. Resultados de simulação confirmam o funcionamento correto do sistema.

## 1 Introdução

A Unidade Lógica e Aritmética (ULA) é um dos principais componentes de sistemas digitais, sendo responsável pela execução de operações matemáticas e lógicas.

Neste trabalho, foi desenvolvida uma ULA de 4 bits com suporte a oito operações, utilizando a linguagem VHDL e implementada em uma FPGA Xilinx Spartan-3. O sistema é controlado por uma máquina de estados finitos (FSM), permitindo a entrada sequencial de dados através de chaves e um botão de confirmação.

O projeto também considera problemas práticos, como o efeito de bouncing em botões mecânicos, garantindo maior robustez no funcionamento do sistema.

## 2 Diagrama da Máquina de Estados (FSM)

A Máquina de Estados Finitos (FSM) implementada é responsável por orquestrar o fluxo de entrada de dados e a exibição de resultados do sistema. Ela dita a sequência cronológica de operações, avançando seus estados majoritariamente através da detecção de uma borda de subida gerada pelo botão de confirmação do usuário (`btn_pressed_edge = 1`).

A FSM é composta por 8 estados principais:

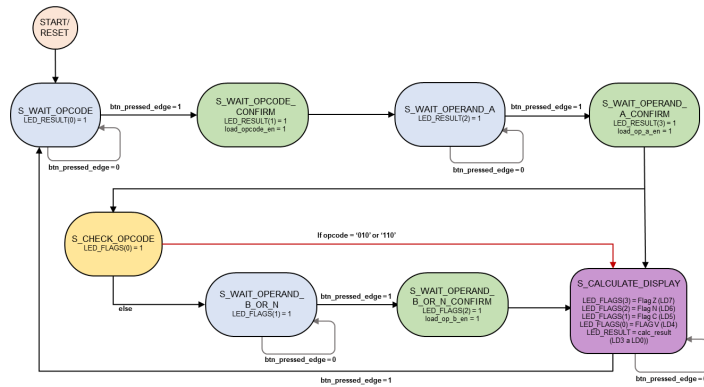


Figura 1: Diagrama de estados da FSM

### 2.1 Descrição dos Estados da FSM

#### 1. S\_WAIT\_OPCODE (Estado Inicial / Espera de Operação)

Este é o estado de repouso ou reset do sistema. Nele, a FSM aguarda que o usuário configure os interruptores (chaves) físicos com o código da operação (Opcode) desejada. Durante este estado, o sistema sinaliza visualmente a etapa atual acendendo o `LED_RESULT(0)`. O sistema permanece travado em um loop de espera (`btn_pressed_edge = 0`) até que o botão de confirmação seja pressionado.

## 2. `S_WAIT_OPCODE_CONFIRM` (Confirmação da Operação)

Estado transitório síncrono. Ao detectar o acionamento do botão, a FSM transita para este estado e ativa o sinal de controle `load_opcode_en = 1`. Isso habilita o registrador correspondente a armazenar o valor inserido nas chaves. O `LED_RESULT(1)` é aceso brevemente. No próximo ciclo de clock, a FSM avança incondicionalmente para o próximo estado.

## 3. `S_WAIT_OPERAND_A` (Espera do Operando A)

Neste estado, o sistema aguarda a inserção do primeiro valor numérico de 4 bits. O `LED_RESULT(2)` permanece aceso indicando a etapa ao usuário. O sistema fica travado neste estado até que o botão de confirmação seja pressionado novamente.

## 4. `S_WAIT_OPERAND_A_CONFIRM` (Confirmação do Operando A)

Similar ao estado de confirmação anterior, este é um estado transitório. A FSM aciona o sinal de controle `load_op_a_en = 1`, permitindo que o valor configurado nos interruptores seja gravado no registrador do Operando A. O `LED_RESULT(3)` acende brevemente antes do avanço incondicional.

## 5. `S_CHECK_OPCODE` (Nó de Decisão / Checagem de Operação)

Este é um estado de roteamento lógico. A FSM avalia o Opcode armazenado na memória para decidir o próximo passo. O `LED_FLAGS(0)` é ativado.

- **Desvio (Atalho):** Se o Opcode corresponder a uma operação unária, especificamente Incremento ("010") ou Negação ("110"), a FSM conclui que não há necessidade de um segundo operando. Dessa forma, ela transita diretamente para o estado final `S_CALCULATE_DISPLAY`.
- **Fluxo Normal:** Para todas as outras operações lógicas e aritméticas (Soma, Subtração, AND, OR, etc.), a FSM transita para `S_WAIT_OPERAND_B_OR_N`.

## 6. `S_WAIT_OPERAND_B_OR_N` (Espera do Segundo Operando)

Estado de espera para a inserção do segundo valor numérico (Operando B). O `LED_FLAGS(1)` acende para guiar o usuário. O avanço ocorre somente mediante o pulso do botão de confirmação.

## 7. `S_WAIT_OPERAND_B_OR_N_CONFIRM` (Confirmação do Segundo Operando)

Estado transitório final de entrada. A FSM emite o sinal `load_op_b_en = 1`, instruindo o banco de registradores a salvar o valor do Operando B. O `LED_FLAGS(2)` acende. A FSM transita incondicionalmente para a exibição de resultados.

## 8. S\_CALCULATE\_DISPLAY (Cálculo e Exibição de Resultados)

Estado final da cadeia de operações. Neste ponto, o sistema estabiliza as saídas e exibe o resultado do processamento da ULA. O driver de saída é instruído a rotear o valor de `calc_result` para os LEDs LD3 a LD0. Simultaneamente, as flags de status da ULA são mapeadas para os LEDs superiores: Flag Z no LD7, Flag N no LD6, Flag C no LD5 e Flag V no LD4.

O sistema entra em um auto-loop, mantendo a exibição na tela (`btn_pressed_edge = 0`) até que o usuário pressione o botão de confirmação novamente, reiniciando o ciclo e retornando a máquina ao estado inicial `S_WAIT_OPCODE`.

## 3 Diagrama de Blocos Estrutural)

O diagrama de blocos ilustra a arquitetura de hardware implementada na FPGA, separando claramente o caminho de dados (*Datath*, representado por linhas sólidas e grossas) do caminho de controle (*Control Path*, representado por linhas tracejadas). O sistema é dividido nos seguintes módulos funcionais:

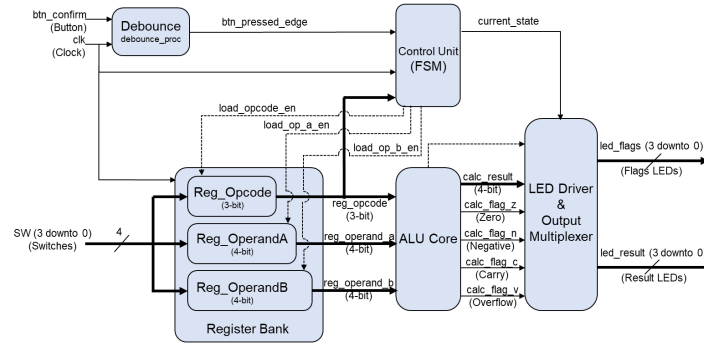


Figura 2: Diagrama de blocos do sistema

### 3.1 Descrição do Diagrama de Blocos Estrutural (Datath e Controle)

#### 1. Entradas Físicas e Sincronismo

O sistema interage com o meio externo através de três portas principais: o clock da placa (`clk`), um botão mecânico (`btn_confirm`) e um barramento de 4 interruptores físicos (`SW(3 downto 0)`). O clock é distribuído simultaneamente para o módulo de *Debounce*, para a Unidade de Controle e para o Banco de Registradores, garantindo o sincronismo de todo o circuito.

#### 2. Módulo de Debounce (`debounce_proc`)

Um bloco essencial para o funcionamento em hardware real. Botões mecânicos geram ruídos (*bouncing*) elétricos ao serem pressionados. Este módulo utiliza o clock do sistema para amostrar o estado do `btn_confirm`. Ele filtra esses ruídos e gera um único pulso limpo de exato um ciclo de clock, denominado `btn_pressed_edge`. Esse sinal é a principal condição de disparo para o avanço dos estados na FSM.

### 3. Unidade de Controle (FSM)

Atua como o “cérebro” do sistema. Este bloco síncrono recebe o pulso limpo do botão (`btn_pressed_edge`) e o clock. Além disso, possui uma linha de realimentação que lê o Opcode do barramento de dados. Com base nisso, a Unidade de Controle emite comandos através de linhas tracejadas:

- **Sinais de Enable:** `load_opcode_en`, `load_op_a_en` e `load_op_b_en`, que autorizam a gravação na memória.
- **Estado Atual:** O barramento `current_state` é enviado ao bloco de saída para informar em qual etapa o processo se encontra.

### 4. Banco de Registradores (Memória Interna)

Composto por três registradores paralelos construídos com lógica síncrona. Eles recebem o barramento contínuo de dados das chaves (SW) em suas entradas. Contudo, os dados só são efetivamente salvos quando a Unidade de Controle envia um pulso para a porta de *Clock Enable* (representada pelas linhas tracejadas). Este banco é formado por:

- **Reg\_Opcode:** Registrador de 3 bits para a operação.
- **Reg\_OperandA:** Registrador de 4 bits para o primeiro número.
- **Reg\_OperandB:** Registrador de 4 bits para o segundo número.

As saídas deste bloco fornecem dados estáveis e contínuos para a unidade de cálculo.

### 5. ALU Core (Núcleo da ULA – Lógica Combinacional)

Este bloco é puramente combinacional. Ele recebe simultaneamente o código da operação e os dois operandos advindos do banco de registradores. Baseado no Opcode de 3 bits, executa instantaneamente as operações aritméticas (soma, subtração, incremento e negação) e lógicas (AND, OR, XOR e SHL) descritas no VHDL.

As saídas deste bloco são:

- **calc\_result:** Barramento de 4 bits contendo o resultado da operação.
- **Flags de Status:** Zero (Z), Negativo (N), Carry/Vai-um (C) e Overflow/Estouro (V).

### 6. LED Driver & Output Multiplexer (Multiplexador de Saída)

Este é o estágio final responsável pela interface com o usuário. O bloco atua como um multiplexador inteligente. Ele recebe os resultados contínuos da ULA, mas também lê o sinal `current_state` vindo da Unidade de Controle.

- Se o sistema estiver nas fases iniciais, o Driver ignora os cálculos da ULA e acende os LEDs de forma posicional, indicando ao usuário qual dado deve ser inserido.
- Quando o sistema atinge o estado final (`S_CALCULATE_DISPLAY`), o Driver roteia o barramento `calc_result` para os LEDs de resultado (`led_result(3 downto 0)`) e mapeia individualmente as quatro flags de status para os LEDs superiores (`led_flags(3 downto 0)`), apresentando a resposta final computada na FPGA.

## 4 Circuito de Debounce

O circuito de *debounce* foi desenvolvido com o objetivo de eliminar os efeitos de *bouncing* mecânico presentes no botão utilizado para confirmação das entradas do sistema. Esse tratamento é fundamental em projetos implementados em FPGA, pois sinais provenientes de dispositivos mecânicos não apresentam transições ideais entre os níveis lógicos.

Quando um botão mecânico é pressionado, seus contatos internos sofrem pequenas vibrações físicas antes de estabilizar completamente. Como consequência, o sinal elétrico alterna rapidamente entre os níveis lógicos ‘0’ e ‘1’ durante alguns milissegundos. Em sistemas digitais síncronos, essas oscilações podem ser interpretadas como múltiplos acionamentos consecutivos, ocasionando erros de funcionamento, avanços indevidos da máquina de estados e gravações incorretas nos registradores.

Para mitigar esse problema, foi implementado um bloco de *debounce* baseado em contagem síncrona utilizando o clock principal da FPGA. O circuito monitora continuamente o estado do botão e somente considera o acionamento válido após o sinal permanecer estável durante um intervalo de tempo previamente determinado.

O limite de estabilidade foi definido pela constante:

```
constant DEBOUNCE_LIMIT : integer := 10000000;
```

O funcionamento do circuito ocorre conforme as seguintes etapas:

- Ao detectar o botão em nível lógico alto, o contador síncrono inicia a contagem dos ciclos de clock;
- Enquanto o sinal do botão permanecer estável, o contador continua sendo incrementado;
- Caso o sinal oscile antes do término da contagem, o contador é reiniciado, descartando o acionamento inválido;
- Quando o contador atinge o valor estabelecido em `DEBOUNCE_LIMIT`, o sistema considera que o botão foi pressionado de maneira válida;
- Após a validação, o circuito gera um único pulso limpo e sincronizado, denominado `btn_pressed_edge`, responsável por controlar o avanço da máquina de estados finitos (FSM).

A utilização desse método aumenta significativamente a confiabilidade do sistema digital, garantindo que cada pressionamento físico do botão produza apenas um único evento lógico. Além disso, o uso de um circuito síncrono baseado em contador torna a solução robusta, simples de implementar em hardware programável e compatível com o restante da arquitetura da FPGA.

## 5 Simulações

### 5.1 Teste 1: Adição (ADD) forçando um Overflow (V)

Como funciona:

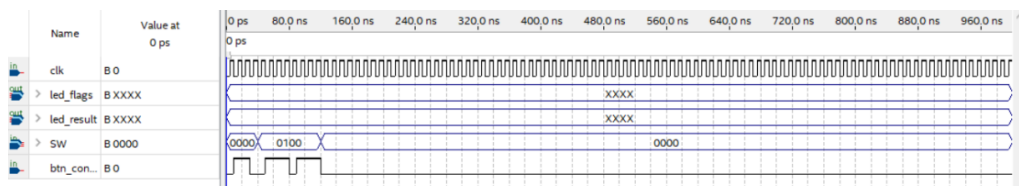
A operação de adição em complemento a 2 utilizando 4 bits suporta apenas números no intervalo de  $-8$  a  $+7$ . Ao realizar a soma  $4+4$ , o resultado matemático correto seria 8, valor que não pode ser representado com 4 bits em complemento a 2. O circuito detecta que dois números positivos produziram um resultado negativo e aciona a flag de Overflow (V).

#### Procedimento realizado:

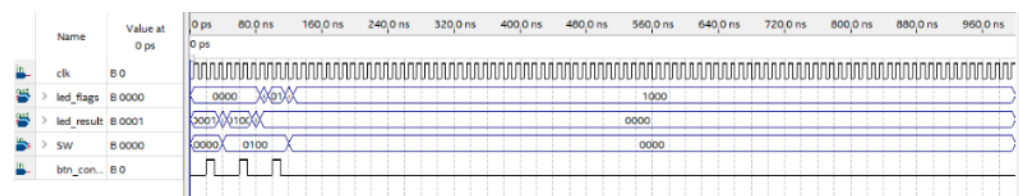
- **Estado 1 (LD0):** Chaves em 000 (Operação ADD). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 0100 (Número 4). Pressionar o botão SUL.
- **Estado 3 (LD5):** Chaves em 0100 (Número 4). Pressionar o botão SUL.

#### Resultado obtido:

- **LEDs da Direita:** 1000 (representação de  $-8$  em complemento a 2).
- **LEDs da Esquerda:** LD4 (Overflow V) e LD6 (Negativa N) ativados.



Input



Output

Figura 3: Simulação da operação ADD com Overflow

## 5.2 Teste 2: Subtração (SUB) forçando a Flag Zero (Z)

#### Como funciona:

A operação realiza a subtração  $A - B$ . Quando todos os 4 bits do resultado são iguais a zero, a flag Zero (Z) é acionada. Neste teste, como os dois operandos possuem o mesmo valor, o resultado é zero.

#### Procedimento realizado:

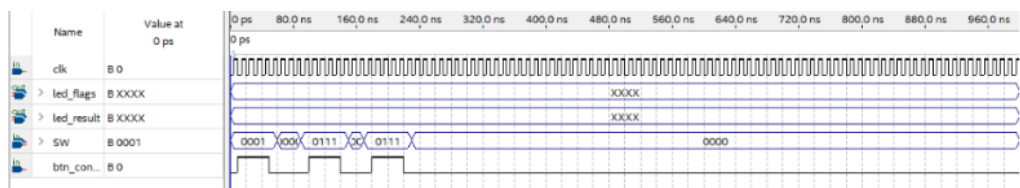
- **Estado 1 (LD0):** Chaves em 001 (Operação SUB). Pressionar o botão SUL.



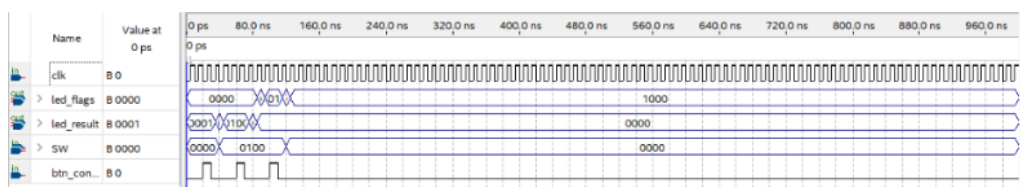
- **Estado 2 (LD2):** Chaves em 0111 (Número 7). Pressionar o botão SUL.
- **Estado 3 (LD5):** Chaves em 0111 (Número 7). Pressionar o botão SUL.

**Resultado obtido:**

- **LEDs da Direita:** Todos apagados (0000).
- **LEDs da Esquerda:** LD7 (Flag Zero Z) ativada.



Input



Output

Figura 4: Simulação da operação SUB com ativação da Flag Zero

### 5.3 Teste 3: Incremento (INC) testando o Pulo da FSM

**Como funciona:**

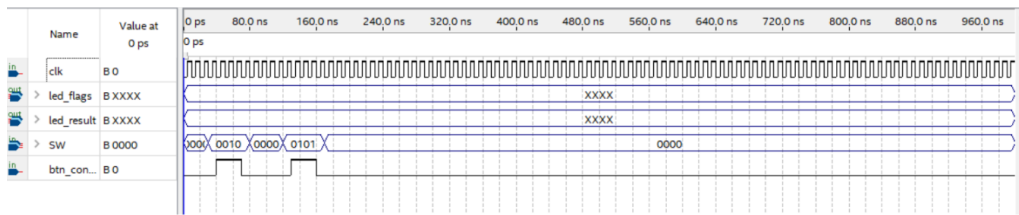
A operação soma 1 ao Operando A. A FSM reconhece o opcode 010 e identifica que esta operação utiliza apenas um operando, pulando a leitura do Operando B diretamente para o cálculo.

**Procedimento realizado:**

- **Estado 1 (LD0):** Chaves em 010 (Operação INC). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 0101 (Número 5). Pressionar o botão SUL.

**Resultado obtido:**

- **LEDs da Direita:** 0110 (Número 6).
- **LEDs da Esquerda:** Todos apagados.



Input



Output

Figura 5: Simulação da operação INC

## 5.4 Teste 4: Porta Lógica OR

**Como funciona:**

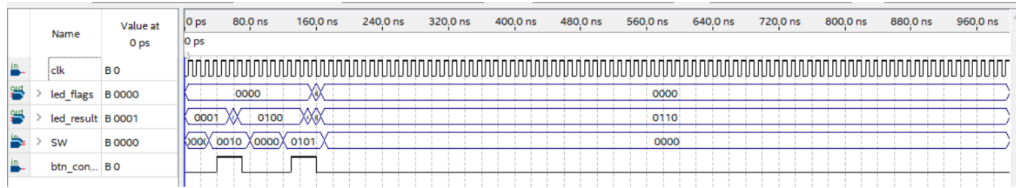
A operação lógica OR compara os bits de A e B. A saída assume 1 se houver pelo menos um bit 1 nas entradas. Flags C e V permanecem zeradas.

**Procedimento realizado:**

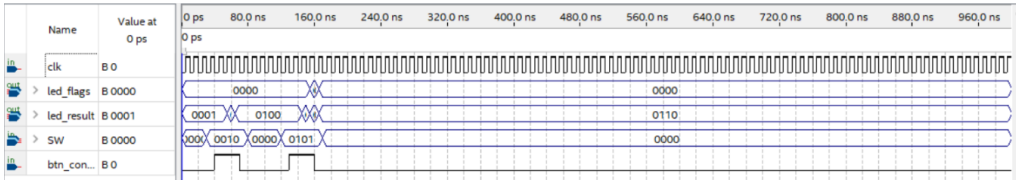
- **Estado 1 (LD0):** Chaves em 011 (Operação OR). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 1010. Pressionar o botão SUL.
- **Estado 3 (LD5):** Chaves em 0101. Pressionar o botão SUL.

**Resultado obtido:**

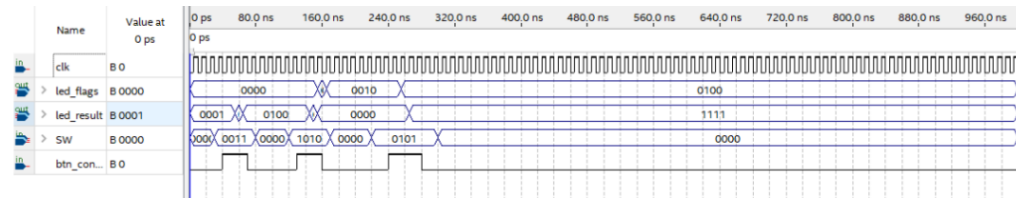
- **LEDs da Direita:** 1111.
- **LEDs da Esquerda:** LD6 (Flag Negativa N) ativada.



Input



Output



Output

Figura 6: Simulação da operação lógica OR

## 5.5 Teste 5: Porta Lógica AND

**Como funciona:**

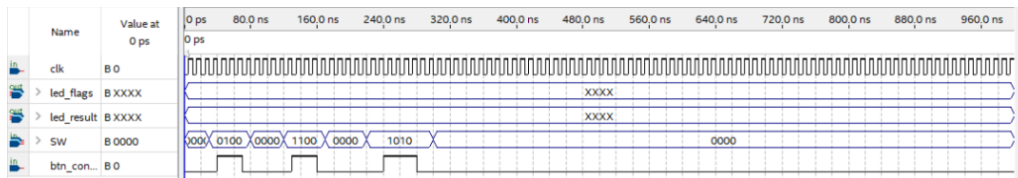
A operação lógica AND resulta em 1 apenas onde ambos os operandos possuírem valor lógico 1.

**Procedimento realizado:**

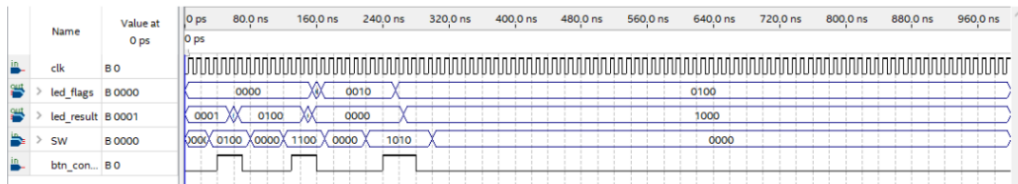
- **Estado 1 (LD0):** Chaves em 100 (Operação AND). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 1100. Pressionar o botão SUL.
- **Estado 3 (LD5):** Chaves em 1010. Pressionar o botão SUL.

**Resultado obtido:**

- **LEDs da Direita:** 1000.
- **LEDs da Esquerda:** LD6 (Flag Negativa N) ativada.



Input



Output

Figura 7: Simulação da operação lógica AND

## 5.6 Teste 6: Porta Lógica XOR (Ou Exclusivo)

**Como funciona:**

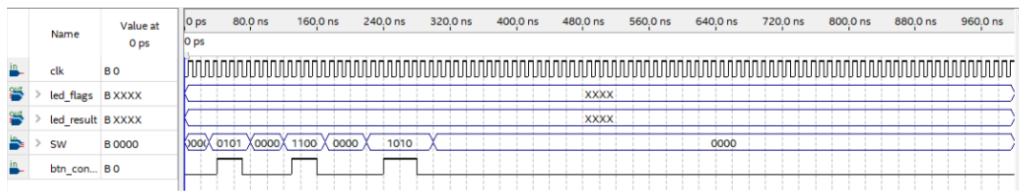
Gera nível lógico 1 apenas onde os bits de A e B forem diferentes.

**Procedimento realizado:**

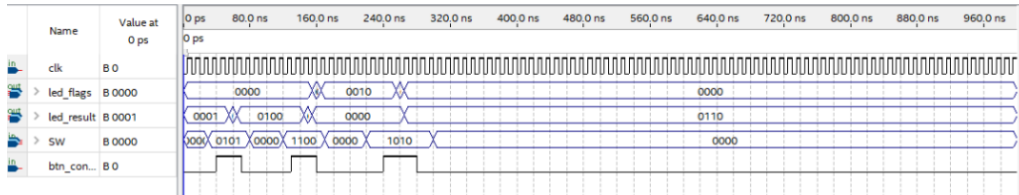
- **Estado 1 (LD0):** Chaves em 101 (Operação XOR). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 1100. Pressionar o botão SUL.
- **Estado 3 (LD5):** Chaves em 1010. Pressionar o botão SUL.

**Resultado obtido:**

- **LEDs da Direita:** 0110.
- **LEDs da Esquerda:** Todos apagados.



Input



Output

Figura 8: Simulação da operação lógica XOR

## 5.7 Teste 7: Negação (NEG) forçando Overflow

**Como funciona:**

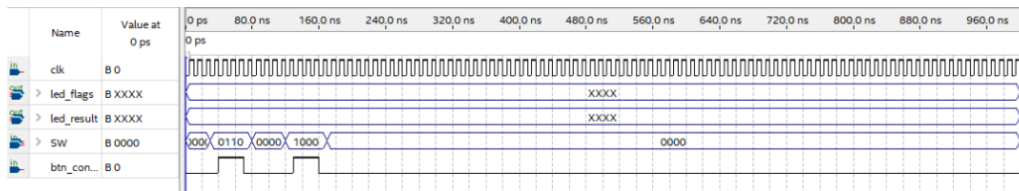
Calcula o oposto via complemento a 2. Tentar inverter o valor  $-8$  causa Overflow, pois  $+8$  estoura a capacidade representável em 4 bits.

**Procedimento realizado:**

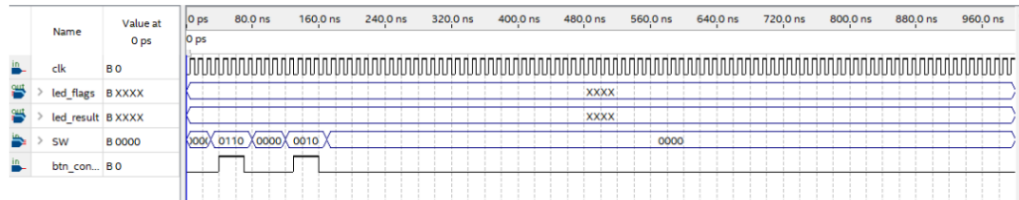
- **Estado 1 (LD0):** Chaves em 110 (Operação NEG). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 1000 (Número -8). Pressionar o botão SUL.

**Resultado obtido:**

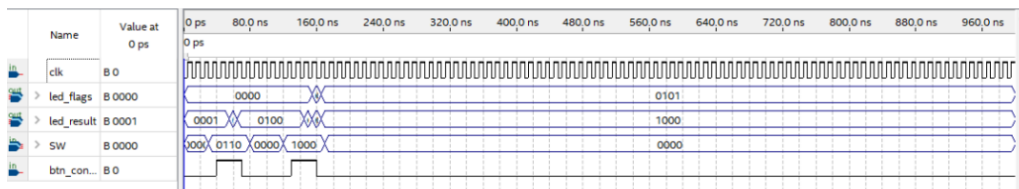
- **LEDs da Direita:** 1000.
- **LEDs da Esquerda:** LD4 (Overflow V) e LD6 (Negativa N) ativados.



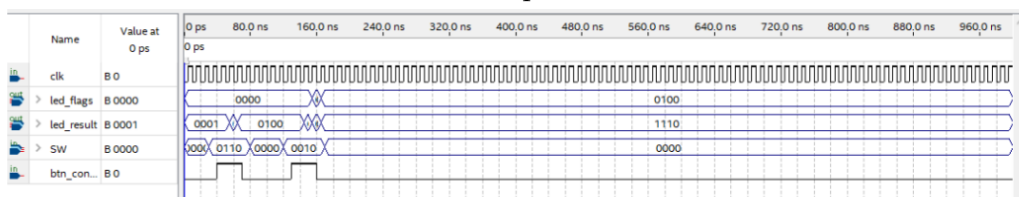
Input



Input



Output



Output

Figura 9: Simulação da operação NEG com Overflow

## 5.8 Teste 8: Shift Left (SHL) testando a Flag Carry (C)

**Como funciona:**

Desloca os bits de A para a esquerda. O bit mais significativo que sai (MSB) é capturado pelo registrador de Carry (C).

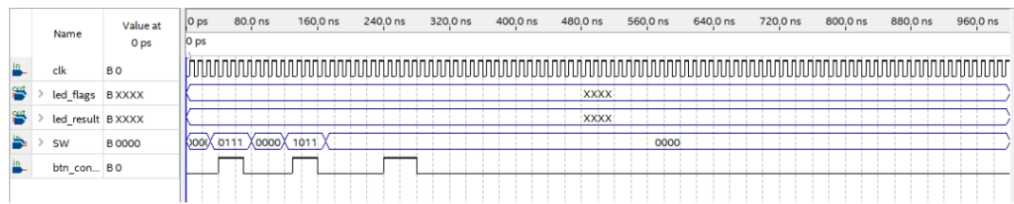
**Procedimento realizado:**

- **Estado 1 (LD0):** Chaves em 111 (Operação SHL). Pressionar o botão SUL.
- **Estado 2 (LD2):** Chaves em 1011. Pressionar o botão SUL.
- **Estado 3 (LD5):** Chaves em 0000. Pressionar o botão SUL.

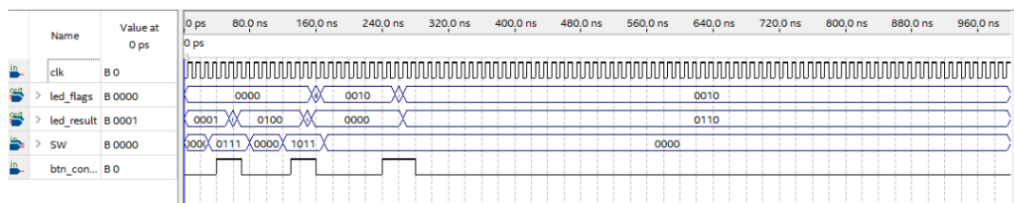
**Resultado obtido:**

- **LEDs da Direita:** 0110.

- **LEDs da Esquerda: LD5 (Flag Carry C) ativada.**



Input



Output

Figura 10: Simulação da operação SHL com ativação da Flag Carry

## 6 Conclusão

A implementação da ULA de 4 bits permitiu aplicar, de forma prática, diversos conceitos fundamentais estudados nas disciplinas de Sistemas Digitais. Durante o desenvolvimento do projeto, foram utilizados conhecimentos relacionados à modelagem de hardware em VHDL, máquinas de estados finitos, registradores, operações aritméticas e lógicas, manipulação de flags e sincronização por clock.

Além disso, a implementação do circuito de *debounce* possibilitou compreender a importância do tratamento de sinais físicos em sistemas digitais reais, aumentando a confiabilidade da interação entre o usuário e a FPGA. O uso da máquina de estados também foi essencial para organizar o fluxo de funcionamento da ULA, tornando o sistema mais estruturado, modular e de fácil entendimento.

Os testes realizados em simulação e na FPGA demonstraram o funcionamento correto das operações implementadas, validando tanto os resultados aritméticos e lógicos quanto o comportamento das flags de status da ULA. A exibição dos resultados e estados por meio dos LEDs permitiu uma verificação visual simples e eficiente do funcionamento do sistema.

Dessa forma, o projeto contribuiu significativamente para o aprendizado prático do grupo, aproximando os conceitos teóricos vistos em sala de aula de uma aplicação real em hardware reconfigurável. A experiência adquirida durante o desenvolvimento também reforçou conhecimentos importantes sobre projeto digital, depuração de circuitos e desenvolvimento de sistemas embarcados utilizando FPGA.

## 7 Referências

Intel Corporation. *Quartus Prime Lite Edition User Guide*.

TOCCI, Ronald J.; WIDMER, Neal S.; MOSS, Gregory L. *Sistemas Digitais: Princípios e Aplicações*. 11. ed. Pearson, 2011.

PEDRONI, Volnei A. *Circuit Design with VHDL*. MIT Press, 2004.

MANO, M. Morris; CILETTI, Michael D. *Digital Design*. 5. ed. Pearson, 2013.

Intel FPGA. *Documentação Oficial FPGA*.

TERASIC Technologies. *DE10-Lite User Manual*.